

Scheduler Implementation Guide

Overview

The scheduler system provides three distinct scheduling policies for parallel execution of BDI graphs: Wavefront, Work Stealing, and Priority-based scheduling.

Scheduler Comparison

Scheduler	Best For	Parallelism	Overhead	Determinism
Wavefront	Regular graphs	High	Low	Yes
Work Stealing	Irregular work-loads	Very High	Medium	No
Priority	Real-time sys-tems	Medium	Low	Yes

Wavefront Scheduler

Concept

Executes nodes in “waves” where each wave contains all nodes whose dependencies have been satisfied.

Wave 0: [A, B, C]

Wave 1: [D, E]

Wave 2: [F]

(no dependencies)

(depend on Wave 0)

(depends on Wave 1)

API

```
WavefrontScheduler* wavefront_scheduler_create(
    BdiGraph* graph,
    DeviceVTable** devices,
    size_t device_count
);

void wavefront_scheduler_free(WavefrontScheduler* sched);

int scheduler_get_next_wavefront(
    WavefrontScheduler* sched,
    Wavefront** out_wavefront
);

int scheduler_execute_wavefront(
    WavefrontScheduler* sched,
    Wavefront* wavefront
);

int wavefront_scheduler_run(WavefrontScheduler* sched);
```

Example

```
#include "kernel/scheduler/wavefront/wavefront_scheduler.h"

// Create scheduler
BdiGraph* graph = /* ... */;
DeviceVTable* devices[] = {&cpu_device_vtable, &gpu_device_vtable};

WavefrontScheduler* sched = wavefront_scheduler_create(
    graph, devices, 2
);

// Execute graph in wavefronts
wavefront_scheduler_run(sched);

// Cleanup
wavefront_scheduler_free(sched);
```

Performance Characteristics

Pros:

- Maximizes parallelism within each wave
- Deterministic execution order
- Low scheduling overhead

Cons:

- Limited by longest dependency chain
- May have idle workers between waves

Work Stealing Scheduler

Concept

Each worker maintains a local queue. Idle workers “steal” work from busy workers’ queues.

```
Worker 0: [A, B, C] → executing A
Worker 1: [D, E]   → executing D
Worker 2: []       → steals C from Worker 0
```

Lock-Free Queue

```
typedef struct {
    atomic_size_t head;
    atomic_size_t tail;
    NodeId items[QUEUE_SIZE];
} LockFreeQueue;

bool queue_push(LockFreeQueue* queue, NodeId node_id);
bool queue_pop(LockFreeQueue* queue, NodeId* out_node_id);
bool queue_steal(LockFreeQueue* queue, NodeId* out_node_id);
```

API

```
WorkStealingScheduler* worksteal_scheduler_create(
    BdiGraph* graph,
    DeviceVTable** devices,
    size_t device_count,
    size_t num_workers
);

void worksteal_scheduler_free(WorkStealingScheduler* sched);

int worksteal_scheduler_run(WorkStealingScheduler* sched);

void worksteal_scheduler_stop(WorkStealingScheduler* sched);
```

Example

```
#include "kernel/scheduler/worksteal/worksteal_scheduler.h"

// Create scheduler with 4 workers
WorkStealingScheduler* sched = worksteal_scheduler_create(
    graph, devices, device_count, 4
);

// Start execution
worksteal_scheduler_run(sched);

// Stop (from another thread)
worksteal_scheduler_stop(sched);

// Cleanup
worksteal_scheduler_free(sched);
```

Performance Characteristics

Pros:

- Excellent load balancing
- Scales well with irregular workloads
- Minimal idle time

Cons:

- Non-deterministic execution order
- Higher overhead from stealing
- Requires careful queue sizing

Priority Scheduler

Concept

Executes nodes based on priority scores, with deadline awareness.

Priority Queue:

1. Node A (priority=100, deadline=1000)
2. Node B (priority=50, deadline=500) ← deadline boost
3. Node C (priority=75, deadline= ∞)

API

```
PriorityScheduler* priority_scheduler_create(
    BdiGraph* graph,
    DeviceVTable** devices,
    size_t device_count
);

void priority_scheduler_free(PriorityScheduler* sched);

int scheduler_set_priority(
    PriorityScheduler* sched,
    NodeId node_id,
    int32_t priority
);

int scheduler_set_deadline(
    PriorityScheduler* sched,
    NodeId node_id,
    uint64_t deadline
);

int priority_scheduler_run(PriorityScheduler* sched);
```

Example

```
#include "kernel/scheduler/priority/priority_scheduler.h"

// Create scheduler
PriorityScheduler* sched = priority_scheduler_create(
    graph, devices, device_count
);

// Set priorities
scheduler_set_priority(sched, 0, 100); // High priority
scheduler_set_priority(sched, 1, 50);  // Medium priority

// Set deadlines
scheduler_set_deadline(sched, 2, 1000); // Must complete by cycle 1000

// Execute
priority_scheduler_run(sched);

// Cleanup
priority_scheduler_free(sched);
```

Priority Calculation

```
int32_t effective_priority(const ScheduledNode* node, uint64_t current_cycle) {
    int32_t priority = node->priority;

    // Boost priority if deadline is near
    if (node->deadline != UINT64_MAX) {
        uint64_t time_to_deadline = node->deadline - current_cycle;
        if (time_to_deadline < 100) {
            priority += 1000; // Urgent!
        }
    }

    return priority;
}
```

Performance Characteristics

Pros:

- Guarantees for high-priority tasks
- Deadline awareness
- Deterministic within priority levels

Cons:

- Lower parallelism than wavefront
- Priority inversion possible
- Requires careful priority assignment

Scheduler Selection Guide

Use Wavefront When:

- Graph has regular structure
- Maximum parallelism is desired

- Deterministic execution is required
- Example: Neural network inference

Use Work Stealing When:

- Workload is highly irregular
- Load balancing is critical
- Non-determinism is acceptable
- Example: Graph traversal, search algorithms

Use Priority When:

- Real-time constraints exist
- Some tasks are more important
- Deadline guarantees are needed
- Example: Control systems, interactive applications

Advanced Topics

Hybrid Scheduling

```
typedef struct {
    WavefrontScheduler* wavefront;
    PriorityScheduler* priority;
} HybridScheduler;

void hybrid_schedule(HybridScheduler* sched, BdiGraph* graph) {
    // Use priority for critical path
    for (size_t i = 0; i < graph->node_count; i++) {
        if (is_critical_path(&graph->nodes[i])) {
            scheduler_set_priority(sched->priority, i, 100);
        }
    }

    // Use wavefront for remaining nodes
    wavefront_scheduler_run(sched->wavefront);
}
```

Dynamic Scheduling

```
void adaptive_schedule(BdiGraph* graph, DeviceVTable** devices) {
    // Profile graph structure
    float parallelism = compute_parallelism(graph);
    float irregularity = compute_irregularity(graph);

    if (parallelism > 0.8f && irregularity < 0.2f) {
        // Regular, parallel → Wavefront
        WavefrontScheduler* sched = wavefront_scheduler_create(/*...*/);
        wavefront_scheduler_run(sched);
    } else if (irregularity > 0.6f) {
        // Irregular → Work Stealing
        WorkStealingScheduler* sched = worksteal_scheduler_create(/*...*/);
        worksteal_scheduler_run(sched);
    } else {
        // Mixed → Priority
        PriorityScheduler* sched = priority_scheduler_create(/*...*/);
        priority_scheduler_run(sched);
    }
}
```

Debugging

Execution Trace

```
void trace_execution(const Scheduler* sched, const GraphNode* node) {
    printf("[%lu] Executing node %llu (opcode=%d) on device %s\n",
        sched->current_cycle,
        node->id,
        node->opcode,
        sched->current_device->name);
}
```

Deadlock Detection

```
bool detect_deadlock(const Scheduler* sched) {
    // Check if all workers are idle but work remains
    bool all_idle = true;
    for (size_t i = 0; i < sched->num_workers; i++) {
        if (!is_worker_idle(sched->workers[i])) {
            all_idle = false;
            break;
        }
    }

    return all_idle && has_pending_work(sched);
}
```

See Also

- Graph Optimization Guide: Preparing graphs for scheduling
- Device Backend API: Execution targets
- HAM Intelligence Guide: Memory-aware scheduling